

Jake DeMuesy

CS 499

6/29/26

Category Two – Narrative

Narratives: Provide a narrative to accompany each artifact in your ePortfolio. The narrative that you write for each artifact should explain why you included the artifact in your ePortfolio and reflect on the process you used to create the artifact. The narrative should focus less on the actual creation of the artifact and more on the learning that happened through the creation of the artifact. Address the following items for each artifact:

Briefly **describe** the artifact. What is it? When was it created?

This artifact is the CS 370 Treasure Hunt deep Q-learning project originally built during Current/Emerging Trends in Computer Science. The premise is a pirate NPC navigating an 8x8 maze to find treasure using a deep Q-learning algorithm. The original submission was a single Jupyter notebook with the Q-training loop, a provided TreasureMaze.py environment class, and a provided GameExperience.py replay buffer -- the main work was implementing the qtrain() function inside the notebook.

The enhanced version split that monolithic structure into a set of discrete, purpose-built Python modules: TreasureMaze.py (the maze environment and state machine), GameExperience.py (experience replay buffer with prioritized sampling), ParameterTuner.py (a hyperparameter search utility with CSV output), and test_parameter_tuner.py (an isolated validation suite). The notebook still exists as the entry point, but the heavy lifting moved out of it.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in software development? How did the enhancement improve the artifact? What specific skills did you demonstrate in the enhancement?

I picked this artifact because it covers two things at once that are hard to demonstrate in a single project -- algorithmic design and software engineering discipline. Deep Q-learning is genuinely interesting from an algorithms standpoint, and the refactor gave me a place to show I can apply those concepts in a structured, maintainable way rather than just getting the notebook to run.

The components that do the most work in terms of showcasing skills are probably GameExperience.py and ParameterTuner.py. GameExperience.py implements prioritized experience replay, which replaced the original uniform random sampling in the replay buffer.

The idea is that not all training transitions are equally useful -- episodes where the model's prediction was way off (high TD error) should get sampled more often than episodes it already understands well. Getting that working correctly meant implementing a probability distribution over the replay buffer based on TD errors, updating those priorities after each training batch, and making sure the double DQN target calculation (using the target network to evaluate actions selected by the main network) was correct.

ParameterTuner.py is the other piece I'd point to. It can run multiple training configurations -- varying learning rate, epsilon decay, discount factor, and priority alpha -- and writes the results to a CSV file for analysis. The CSV works as a persistent log of each run: input hyperparameters and output performance markers (win rate, final loss, epochs to convergence, wall-clock time) all land in one record per run, which makes cross-run comparison pretty straightforward. Keeping the input config and output metrics together in the same row means the file is self-describing without needing a separate config log alongside it.

The test suite in test_parameter_tuner.py uses stub implementations of build_model() and qtrain() to validate the tuner's CSV output format and config injection logic without running a real TensorFlow training loop. Including that was deliberate -- it demonstrates that I thought about regression testing!

A bug fix also ended up being part of this enhancement, though it wasn't in the original plan. The original GameExperience.py had a duplicate __init__ method -- the second definition silently overwrote the first, which meant priority_alpha, priority_epsilon, and the priorities list never actually got initialized. The entire PER system was structurally broken without it being obvious from a quick read. Finding and fixing that was part of the enhancement work.

Reflect on the process of enhancing the artifact. What did you learn as you were creating it and improving it? What challenges did you face? How did you incorporate feedback as you made changes to the artifact? How was the artifact improved? Which course outcomes did you partially or fully meet with your enhancements? Which do you feel were not met?

I think after my reflection, I had a similar experience with this enhancement as I did with my previous project -- the difference between what "works well" and what is "well-designed" are completely different. The original notebook ran fine -- the pirate found the treasure, the training loop completed, no errors. But the GameExperience class had that duplicate __init__ bug, which meant the prioritized experience replay wasn't actually doing anything. The code looked right at a glance but the second constructor was silently dropping the priority-related parameters on every instantiation. That's the kind of thing that's easy to miss in a notebook environment where you're mostly focused on whether the output looks reasonable.

The modularization work was pretty straightforward once I had a clear picture of where the boundaries should be. TreasureMaze.py is the state-machine provider -- it owns the grid,

handles transitions, and reports game status. GameExperience.py is the orchestrator on top of that -- it stores transitions, manages the replay buffer, and builds the training batches. ParameterTuner.py sits above both of them as an experiment controller. Those roles don't overlap much, which is what made the separation feel natural rather than forced. The Agent-Environment loop Dr. Bolton mentioned is basically the relationship between TreasureMaze and GameExperience -- the environment is stateful and deterministic given an action, the agent is responsible for learning a policy. Naming that explicitly in the module comments was a small addition but it makes the architecture easier to follow for anyone reading the code cold!

The feedback from Dr. Bolton pushed me to be more specific about a couple things I'd left implicit. The CSV logging in ParameterTuner.py is a real data management decision -- flat-file storage with a defined schema, written once after all configs complete so a failed mid-batch run doesn't produce a partial file. I hadn't explained any of that in the original narrative, I'd just described what the file contained. Adding the rationale for those choices (and documenting the valid parameter ranges so external developers know where the bounds are) made the write-up a lot more defensible.

In terms of course outcomes, this enhancement covers the algorithms and data structures outcome pretty directly -- the PER implementation is a real algorithmic choice with documented trade-offs, not just a code restructure. The software engineering outcome is covered through the modularization and the test suite. The professional communication outcome runs through the narrative and the module-level comments. The security outcome isn't really addressed here -- that was handled in Enhancement Three with the dashboard RBAC work. The database outcome is also Enhancement Three territory. So this one is strong on algorithms and engineering, lighter on everything else!